

第二章：词法分析

DFA——确定有限自动机

内容介绍

- ◆ 确定有限自动机（DFA）的定义
- ◆ DFA的表示
- ◆ DFA接受的集合
- ◆ 应用实例
- ◆ 用DFA描述单词
- ◆ 自动机的实现
- ◆ 自动机等价

有限自动机Finite Automaton



- ◆ 工作过程：在初始状态下，读头指向输入带的最左单元，准备读入第一个字符，每读入一个字符，从当前状态进入下一状态；当读头读入所有字符后，若状态进入终态，则输入带上的输入串被接受；否则，输入串有错。

1. 确定有限自动机的定义DFA

确定有限自动机M为一个五元组：

$$M = (S, \Sigma, S_0, f, Z), \text{ 其中:}$$

1. S 是一个有穷状态集，它的每个元素称为一个状态；
2. Σ 是一个有穷字母表，它的每个元素称为一个输入字符；
3. $S_0 \in S$ ，是唯一的一个初始状态(开始状态)；
4. f 是状态转换函数： $S \times \Sigma \rightarrow S$ ，且单值函数. $f(S_i, a) = S_k$ 表示：当前状态为 S_i ，遇输入字符 a 时，自动机将唯一地转换到状态 S_k ，称 S_k 为 S_i 的一个后继状态；
5. $Z \subseteq S$ ，是终止状态集（可接受状态集、结束状态集），其中的每个元素称为终止状态（可接受状态、结束状态）， Z 可空.

1. 确定有限自动机的定义

◆ DFA确定性的体现：

1. 初始状态唯一
2. 状态转换函数 $f: S \times \Sigma \rightarrow S$ 是一个单值函数，也就是说，对任何状态 $s \in S$, 和输入符号 $a \in \Sigma$, $f(s, a)$ 唯一地确定了下一个状态，即至多确定一个状态
3. 转换边上不能标 ϵ ，即不接受没有任何输入就进行状态转换的情况

1. 确定有限自动机的定义

- ◆ 例1，确定有限自动机 $M_1 = \{S, \Sigma, S_0, f, Z\}$ ，其中元素定义如下：

$S = \{0, 1, 2\}$,

$\Sigma = \{a, b\}$,

$S_0 = 0$,

$f(0, a) = 1, f(0, b) = 2, f(1, a) = 2$,

$Z = \{2\}$

2. DFA的表示

状态转换矩阵

- ❖ 用 Σ 中的字符表示列标
- ❖ 用状态来表示行标
- ❖ 矩阵元素表示自动机的状态转换函数
- ❖ 标识初始状态和终止状态：一般约定，第一行表示开始状态 S_0 ，或在右上角标注“+”；右上角标有“*”或“-”的状态为终止状态

例1中DFA M1的状态转换矩阵表示如表1或表2所示：

表1

输入字符 状态	a	b
0	1	2
1	2	
2*		

默认第一行表示开始状态

表2

输入字符 状态	a	b
1	2	
0 ⁺	1	2
2 ⁻		

用+标记开始状态

2. DFA的表示

- ◆ DFA $M2 = (\{T, U, V, Q\}, \{a, b\}, f, T, \{Q\})$, 其中 f 定义为:

$f(T, a) = U$

$f(T, b) = V$

$f(V, a) = U$

$f(V, b) = Q$

$f(U, a) = Q$

$f(U, b) = V$

$f(Q, a) = Q$

$f(Q, b) = Q$

输入字符 状态	a	b
T	U	V
U	Q	V
V	U	Q
Q^*	Q	Q

2. DFA的表示

状态转换图：用有向图表示自动机

1. 结点表示状态：

1.1 非终止状态由单圆圈围住的状态标识来表示；

1.2 终止状态由双圆圈围住的状态标识来表示（或由单圆圈围住的状态标识并标识以“-”来表示）；

1.3 开始状态由一个箭头指向的状态结点来表示(或标识以“+”来表示).

2.有向边表示状态转换函数，若 $f(S_i, a) = S_k$ ，则由表示 S_i 的状态结点到表示 S_k 的状态结点发出一条标识为 a 的有向边.

- ◆ DFA $M2 = (\{T, U, V, Q\}, \{a, b\}, f, T, \{Q\})$, 其中 f 定义为:

$$f(T, a) = U$$

$$f(T, b) = V$$

$$f(U, a) = Q$$

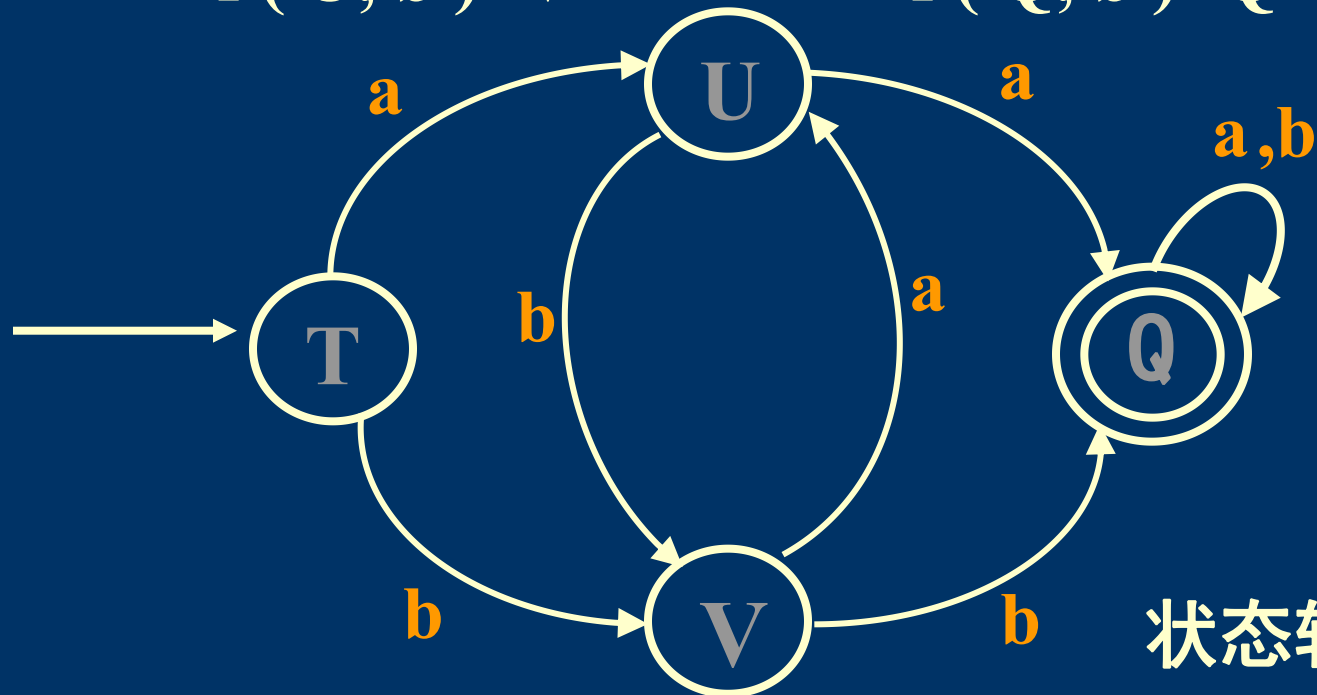
$$f(U, b) = V$$

$$f(V, a) = U$$

$$f(V, b) = Q$$

$$f(Q, a) = Q$$

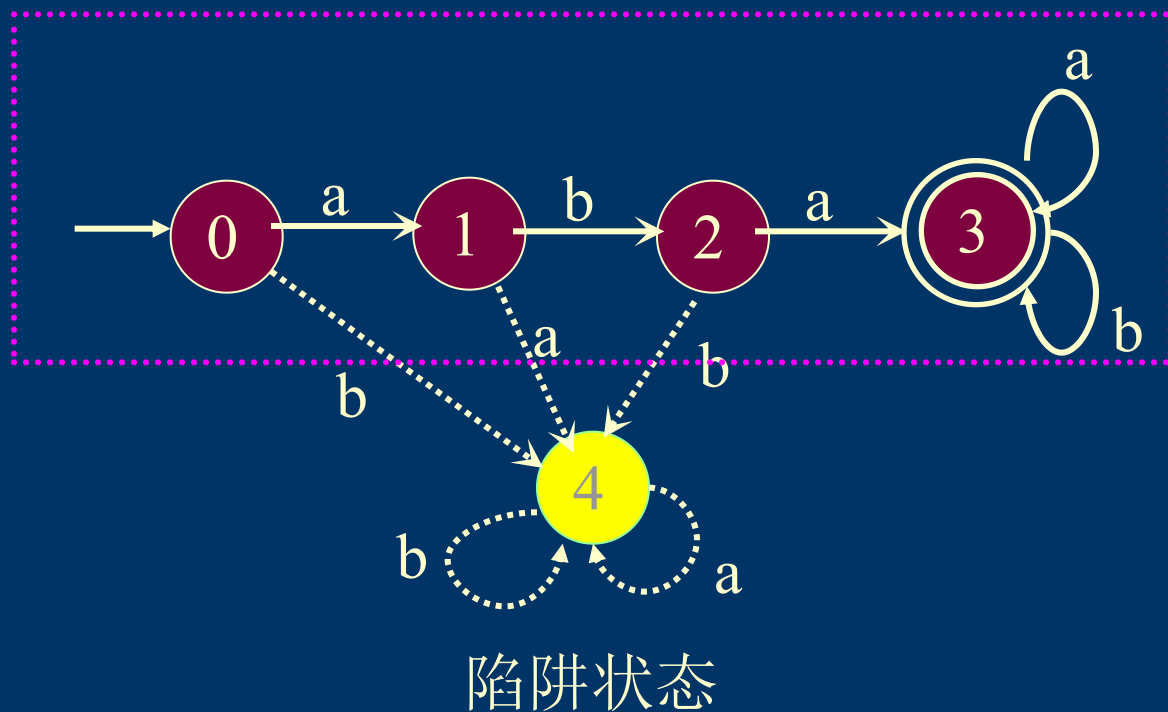
$$f(Q, b) = Q$$



状态转换图

DFA的两种表示形式

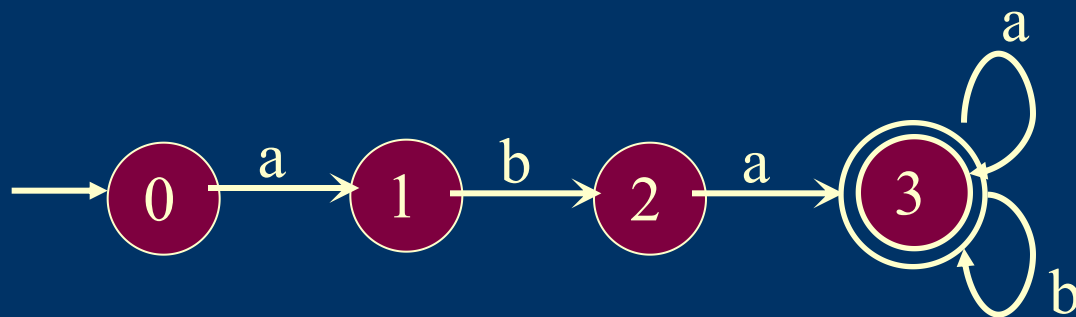
	a	b
0	1	4
1	4	2
2	3	4
3*	3	3
4	4	4



DFA的两种表示形式

- ◆ 去除陷阱状态
- ◆ 缺省状态: $\perp$
- ◆ 缺省状态转换函数: $f(0,b) = \perp$ 或空

	a	b
0	1	$\perp$
1	$\perp$	2
2	3	$\perp$
3*	3	3

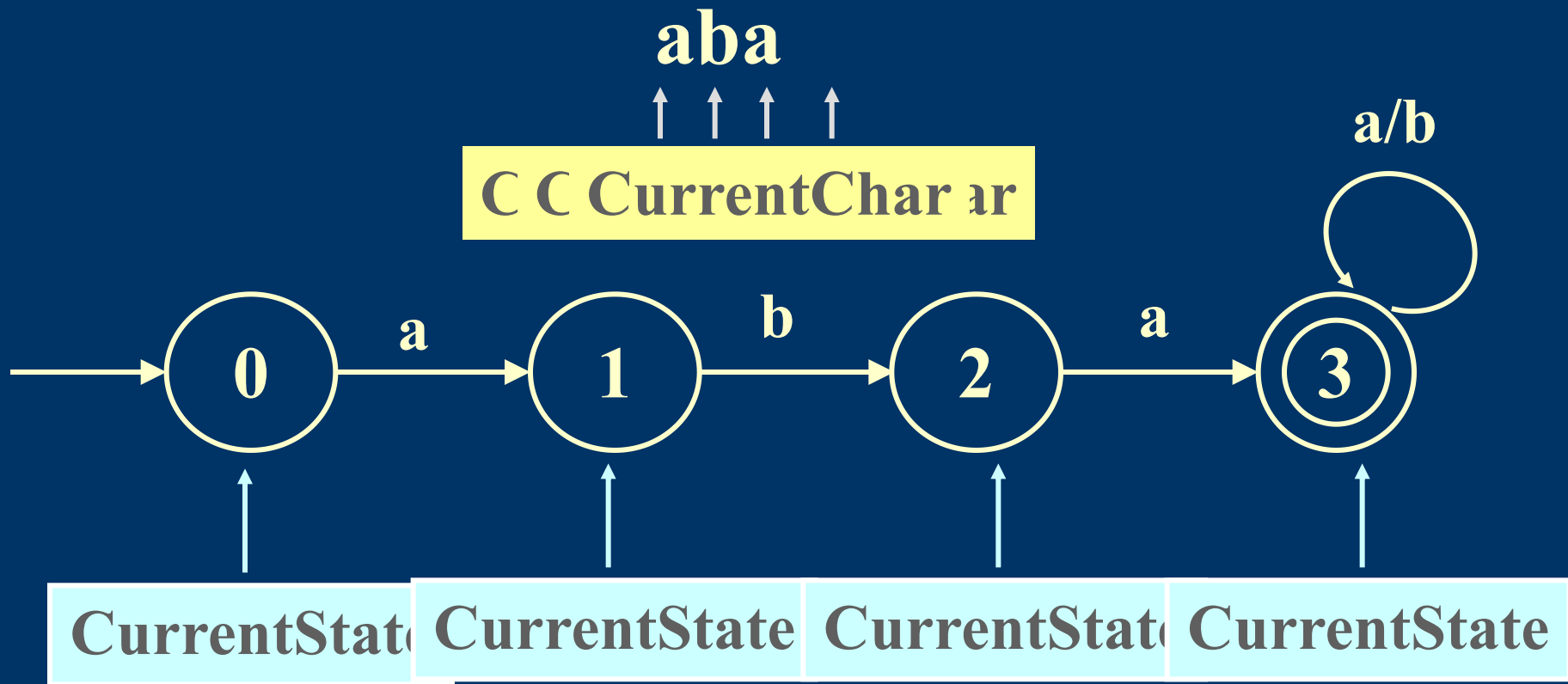


利用陷阱状态: 表示错误处理状态

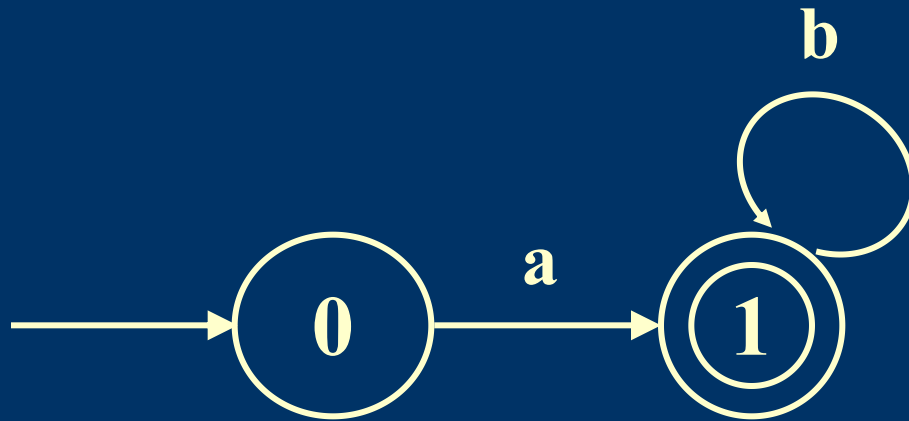
3. DFA接受的串

- ◆ 对于 Σ^* 中的任何字符串 α , 若存在一条从初始结点到某一终止结点的路径, 且这条路上所有弧的标记符连接成的字符串等于 α , 则称 α 可为DFA M 所接受 (识别)。
- ◆ DFA M 所能接受的字符串的全体, 称为自动机所能识别的语言, 记为 $L(M)$ 。
- ◆ 特别地, 若DFA M 的初始状态同时又是终止状态, 则空字符串可为DFA M 所接受 (识别)。

例: $L(M_1) = \{ aba, abaa, abab, abaab, \dots \}$

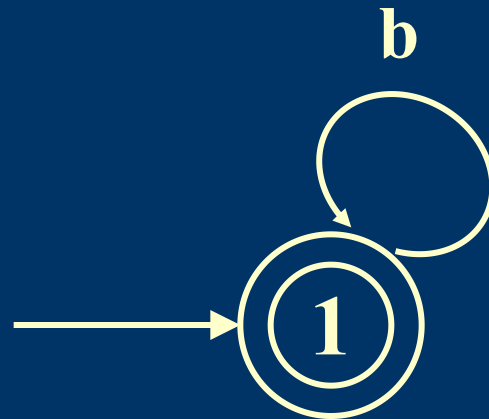


例: $L(M_2) = \{a, ab, abb, abbb, \dots\}$



DFA M_2

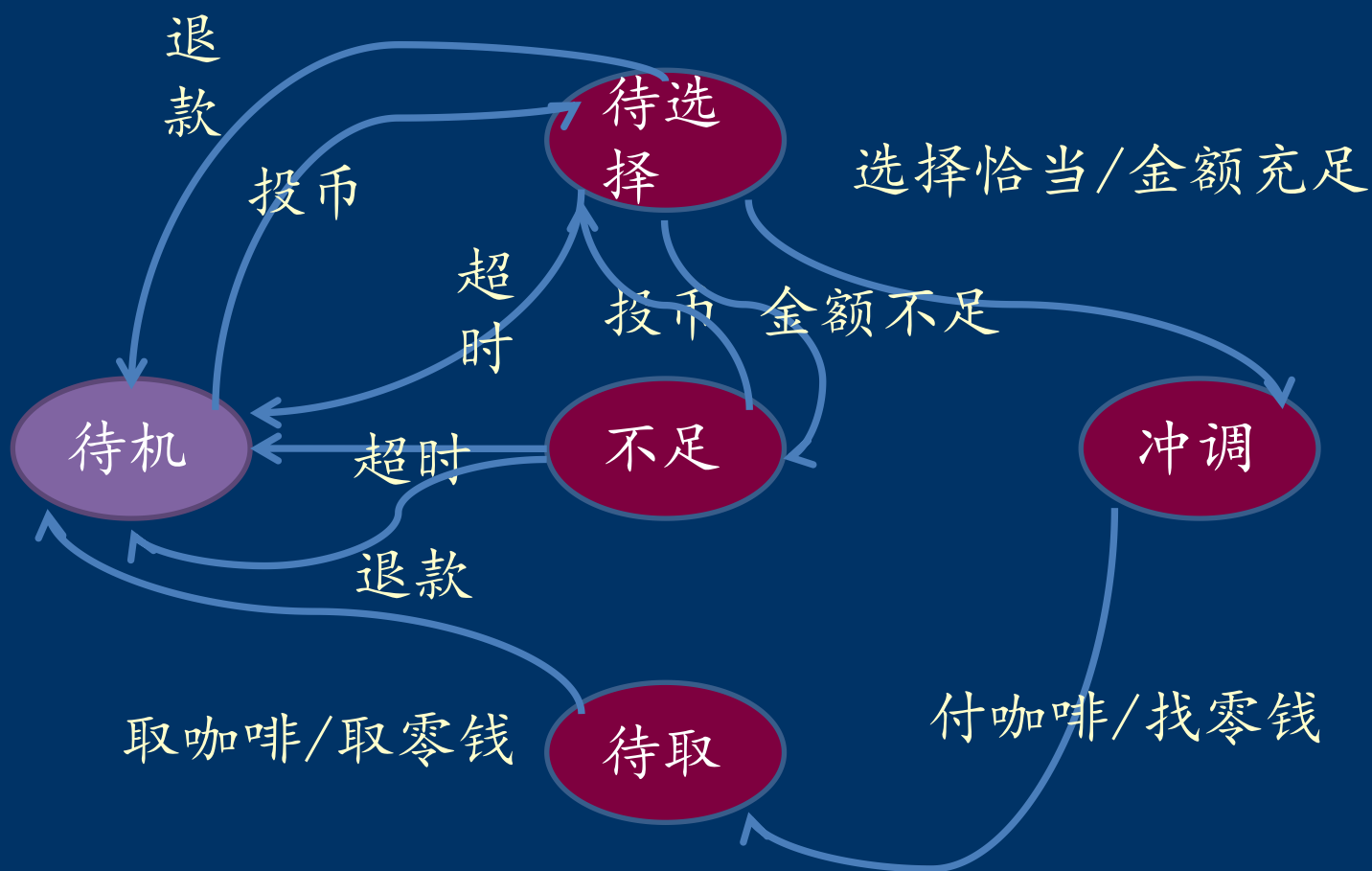
例： $L(M_3) = \{\epsilon, b, bb, bbb, \dots\}$



DFA M_3

4. 应用实例

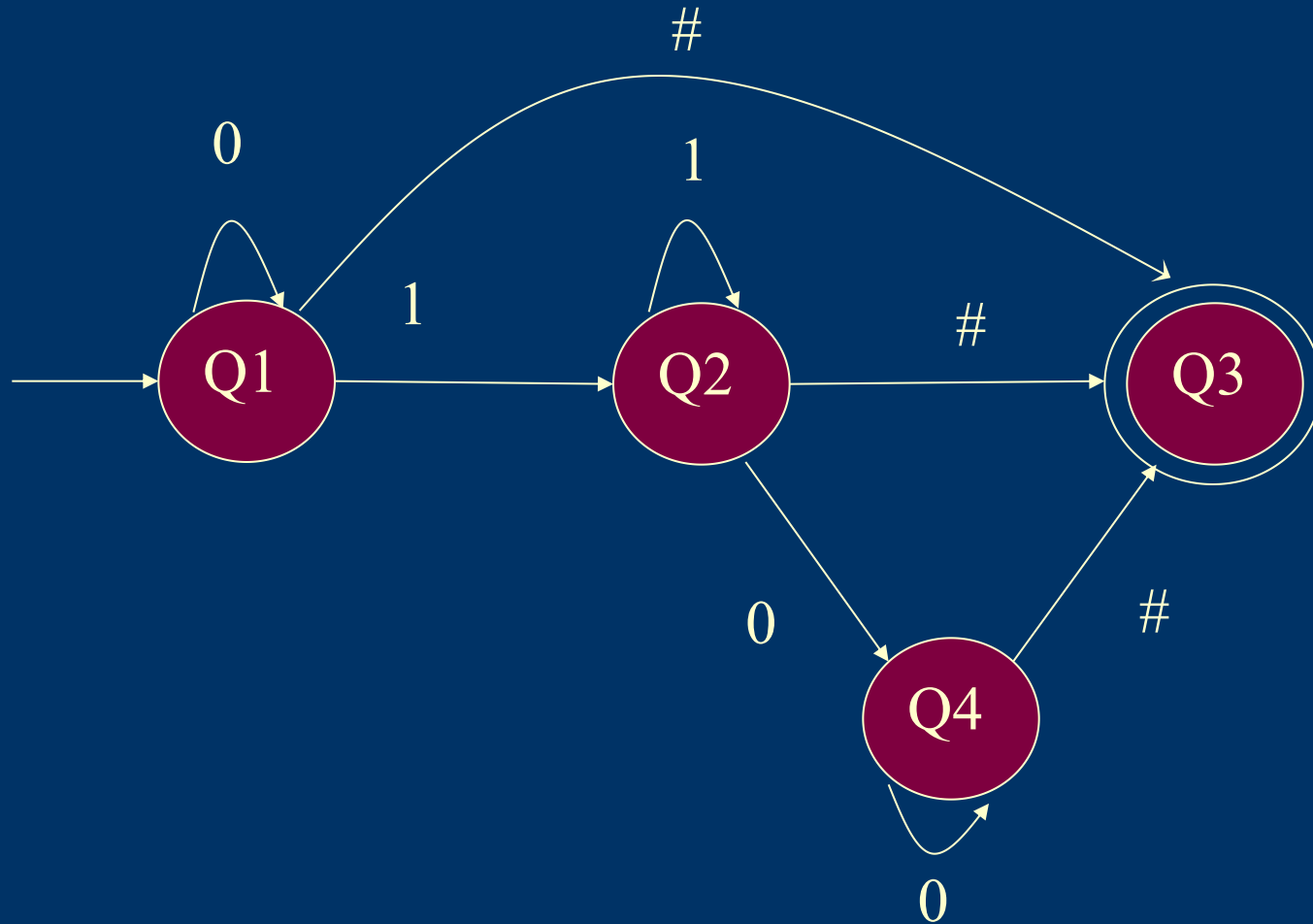
—描述离散状态的系统



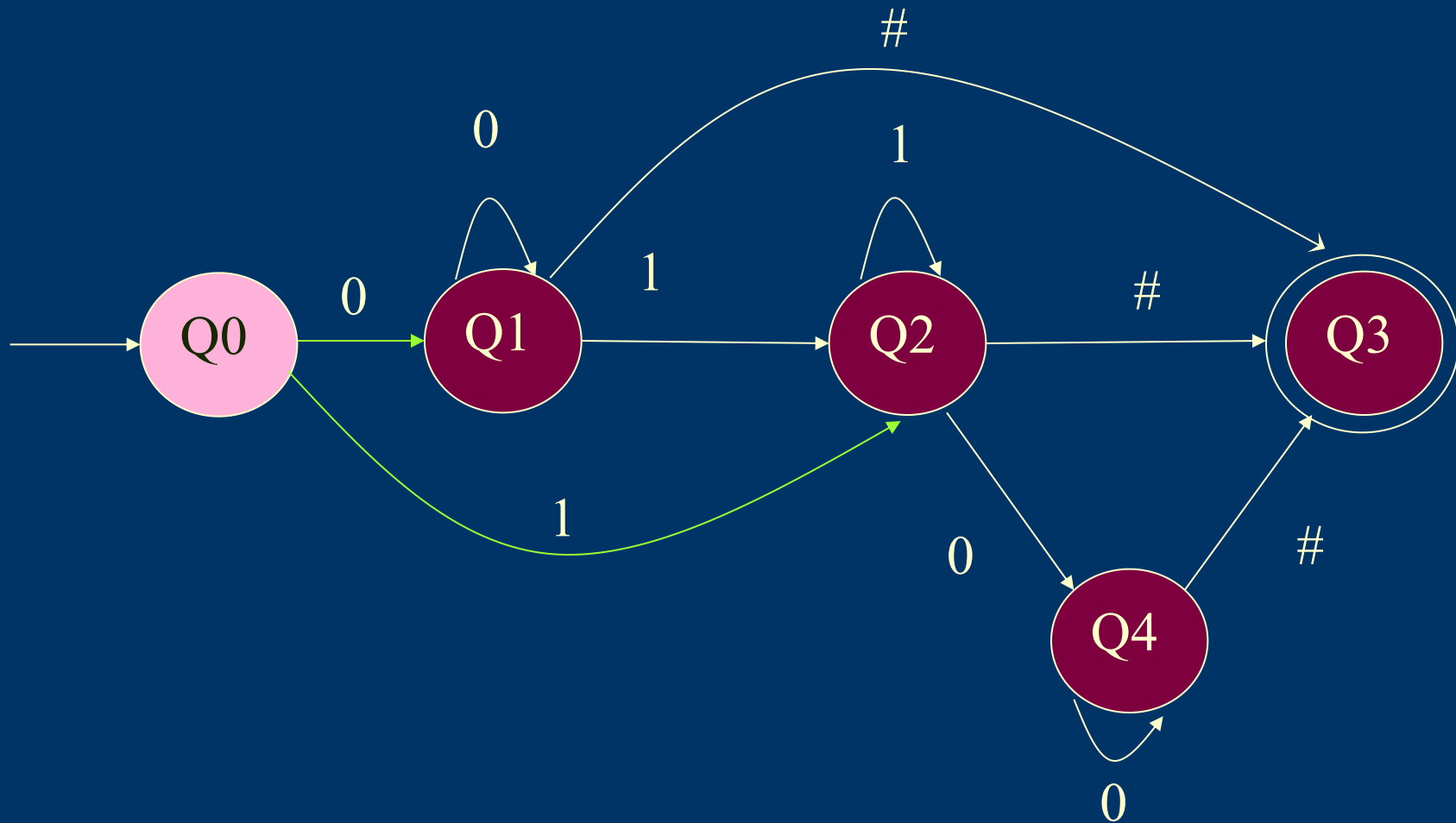
4. 应用实例

- ◆ 编写一个程序，对任意输入的一个字符串，判断其是否合法，对字符串的具体要求如下：
 - 1) 这个字符串由0, 1组成，以#作为结尾
 - 2) 合法的要求是不能出现两个1串。如0101#就是非法的，011#是合法的。

接受{#, 0#, 1#, 00...#, 11...#, 0...01...1#, 0...01...10...0#}

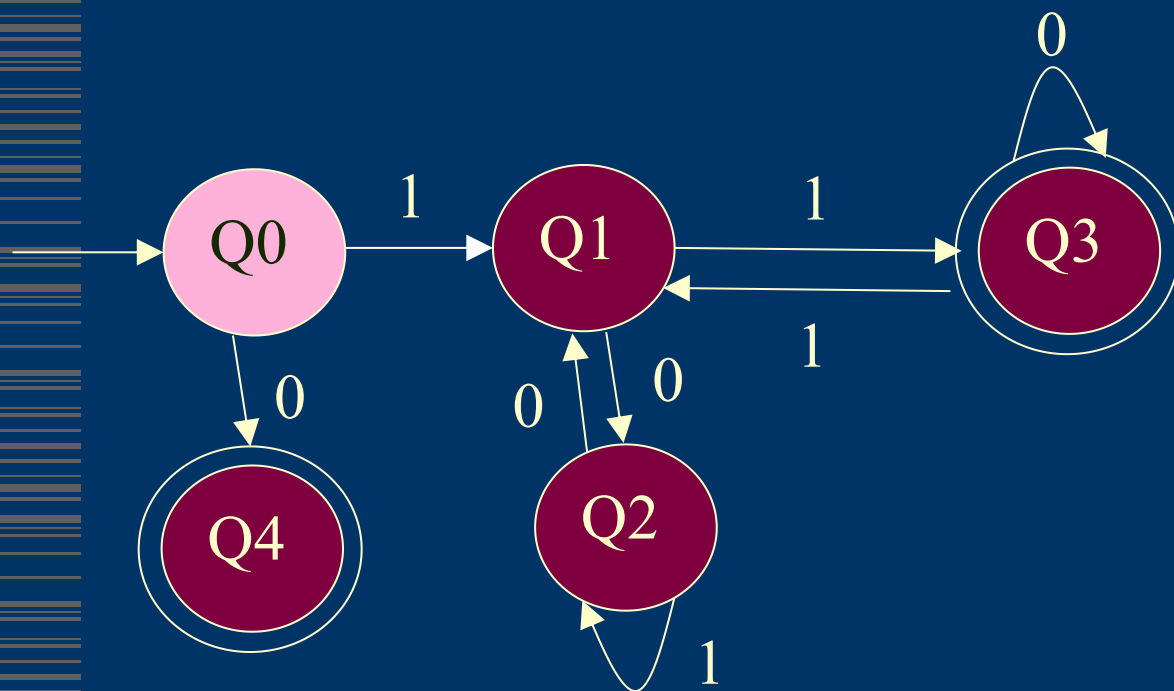


接受 $\{0\#, 1\#, 00\dots\#, 11\dots\#, 0\dots 01\dots 1\#, 0\dots 01\dots 10\dots 0\#\}$



4. 应用实例

- ◆ 给出一个描述能被3整除的合法二进制数（不含先导0）的DFA。

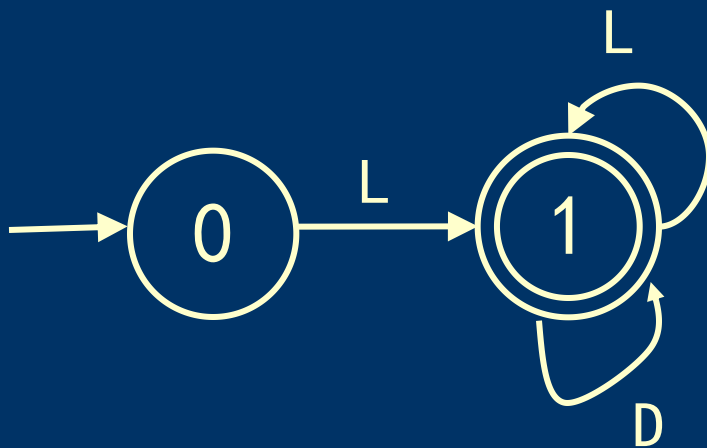


二进制数模3的余数只有0, 1, 2三种状态;
二进制左移一位= $\times 2$,
末位加0/1;
转移: 新余数 = (旧余数 $\times 2$ + 新位) mod 3

5. DFA描述单词

◆ 标识符的描述

我们用L表示所有字母，D表示数字，则有：



5. DFA描述单词

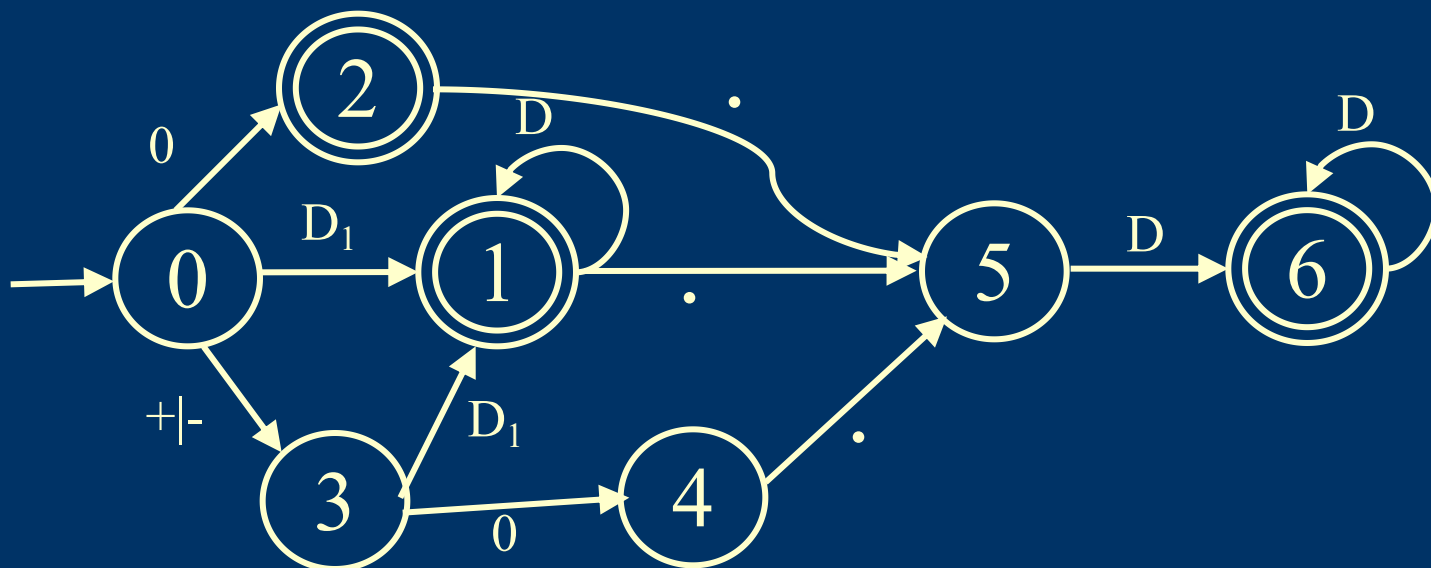
◆ 常数的描述

$D_1 = 1 | 2 | 3 | \dots | 9$

无符号整数

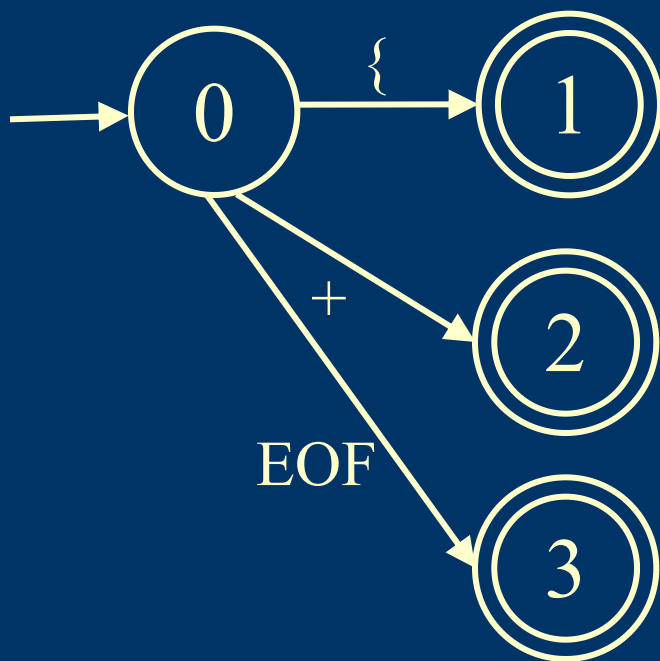
带符号整数

实数



5. DFA描述单词

◆ 特殊符号



6. 自动机的实现

- ◆ 直接转换法

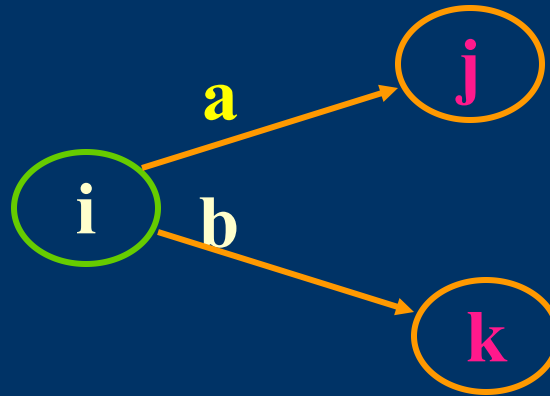
每个状态对应一个带标号的switch语句，转向边对应goto语句。

- ◆ 状态转换矩阵

自动机存储在矩阵中，状态比较多，每次都去查表，跳转。

直接转换法的实现

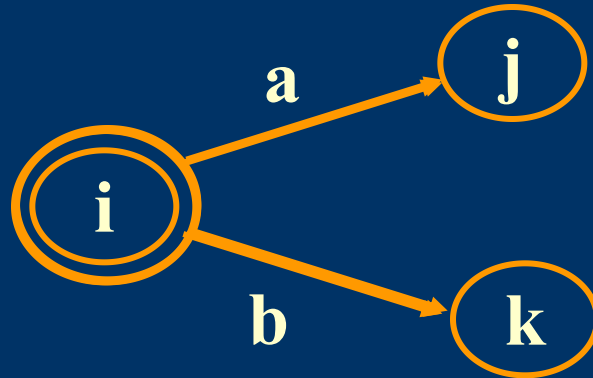
1. 非终止状态对应的switch语句



```
Li: switch ( CurrentChar )  
{ case 'a' : NextChar(); goto Lj;  
  case 'b' : NextChar(); goto Lk;  
  default  : Error();  
}
```

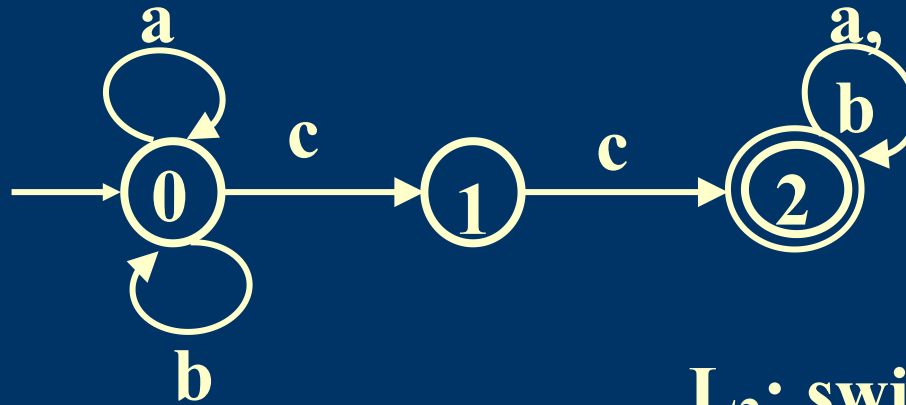
直接转换法的实现

2. 终止状态对应的switch语句



```
Li: switch ( CurrentChar )  
{ case 'a' : NextChar(); goto Lj;  
  case 'b' : NextChar(); goto Lk;  
  case 'EOF' : Accept();  
  default   : Error();  
}
```

直接转换法实现的例子



```
L0: switch ( CurrentChar )  
{ case ' c' : Nextchar();goto L1;  
  case ' a' : Nextchar();goto L0;  
  case ' b' : Nextchar(); goto L0;  
  default  : Error(); }
```

```
L1: switch ( CurrentChar )  
{ case ' c' : Nextchar();goto L2;  
  default  : Error(); }
```

```
L2: switch ( CurrentChar )  
{ case ' a' :  
  Nextchar();goto L2;  
  case ' b' :  
  Nextchar(); goto L2;  
  case 'EOF': Accept();  
  default  : Error();  
}
```

状态转换矩阵的实现

1. 当前状态`State`置为初始状态；
2. 读一个字符 $\rightarrow$ `CurrentChar`；
3. 如果`CurrentChar` $\neq$ `Eof`并且

$T(State, CurrentChar) \neq error$ ，

则当前状态转为新的状态：

$State = T(State, CurrentChar)$ ，

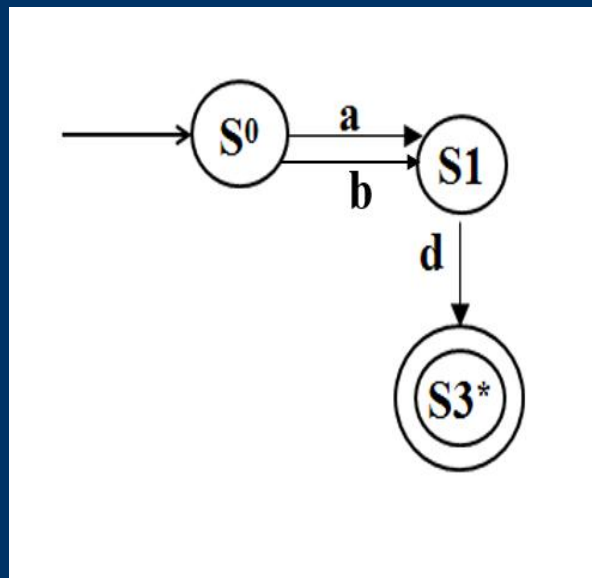
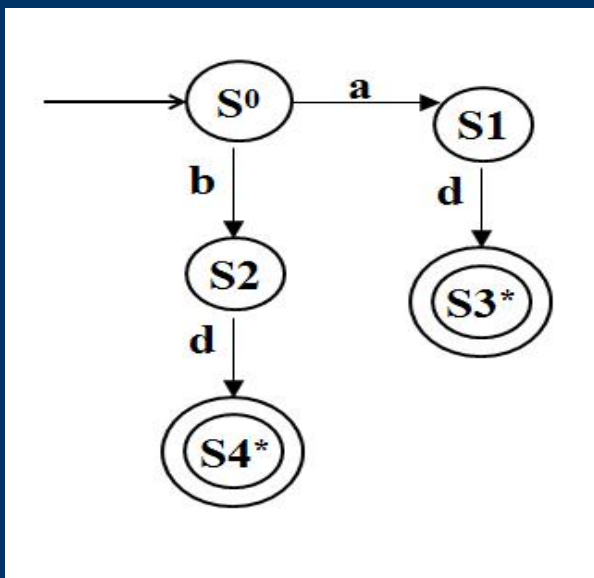
读下一字符 $\rightarrow$ `CurrentChar` ；

重复第3步工作.

4. 如果当前字符为`EOF`并且当前状态属于终止状态，则接受当前字符串，程序结束；否则报错.

7. 自动机等价

- 对于两个DFA M_1 和 M_2 , 若有 $L(M_1) = L(M_2)$ 则称 M_1 和 M_2 等价



农夫过河问题

- ◆ 一个人带着狼、山羊和白菜在一条河的左岸。有一条船，大小正好能装下这个人和其他三件东西中的一件。人和他的随行物都要过到河的右岸。人每次只能将一件东西摆渡过河。但若人将狼和羊留在同一岸而无人照顾的话，狼将把羊吃掉。类似地，若羊和白菜留下来无人照看，羊将会吃掉白菜。请问是否有可能渡过河去，使得羊和白菜都不被吃掉？如果可能，请用有限自动机写出渡河的方法。

